

Reviewer Pack — Symmetric Group Fourier Coefficient Sum Distinguishes Read-T...

Entry 399f95beac0c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 10:06:11 UTC

Symmetric Group Fourier Coefficient Sum Distinguishes Read-Twice from Read-Once BPs

Entry ID: 399f95beac0c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 10:06:11 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For a read-twice BP P over n variables, the sum of absolute values of its Fourier coefficients in the symmetric group representation is $\Omega(2^{\{n/2\}})$, whereas for read-once BPs it is $O(n^2)$.

Rationale (proposer’s reasoning):

The symmetric group’s representation theory captures permutation symmetries inherent in read-twice BPs, which are absent in read-once. Fourier coefficients encode these symmetries, creating a gap exploitable for separation.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dd28157c74aaa7e2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_read_twice_bp(n):
    return [[random.choice([0, 1]) for _ in range(2)] for _ in range(n)]

def generate_read_once_bp(n):
    return [random.choice([0, 1]) for _ in range(n)]

def fourier_transform(bp, n):
    def gaussian_elimination(A, b):
        m, n = len(A), len(b)
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i+1, m):
                factor = Fraction(A[j][i], A[i][i])
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
                    b[j] -= factor * b[i]
        return [b[i] / A[i][i] for i in range(m)]

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def identity_matrix(n):
        I = [[Fraction(1 if i == j else 0) for j in range(n)] for i in range(n)]
        return I
```

```

def inverse_matrix(A):
    n = len(A)
    A_augmented = [A[i] + [Fraction(1 if i == j else 0) for j in range(n)] for i in range(n)]
    identity = identity_matrix(n)
    gaussian_elimination(A_augmented, identity)
    return [row[n:] for row in A_augmented]

def tensor_product(A, B):
    m, n = len(A), len(B[0])
    p, q = len(B), len(B[0][0])
    C = [[0] * q for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                for l in range(q):
                    C[i][j][l] += A[i][k] * B[k][j][l]
    return C

def kronecker_delta(i, j):
    return Fraction(1 if i == j else 0)

def symmetric_group_representation(n):
    def sgn(permutation):
        n = len(permutation)
        inversions = 0
        for i in range(n):
            for j in range(i+1, n):
                if permutation[i] > permutation[j]:
                    inversions += 1
        return Fraction(1 if inversions % 2 == 0 else -1)

    def apply_permutation(bp, permutation):
        return [bp[permutation[i]] for i in range(len(bp))]

    representations = []
    for n_cycles in range(1, n+1):
        for cycle in itertools.permutations(range(n), n_cycles):
            representation = [0] * n for _ in range(n)]
            for i in range(n):
                representation[apply_permutation(cycle, [i])[0]][i] = sgn(cycle)
            representations.append(representation)
    return representations

def fourier_coefficient(bp, representation):
    n = len(bp)
    m = len(representation)
    A = [[0] * m for _ in range(m)]
    b = [0] * m
    for i in range(m):
        for j in range(n):
            A[i][j] = representation[j][i]
            b[i] += bp[j]
    return gaussian_elimination(A, b)[0]

representations = symmetric_group_representation(n)
coefficients = [fourier_coefficient(bp, rep) for rep in representations]

```

```

    return sum(abs(coeff) for coeff in coefficients)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        read_twice_bp = generate_read_twice_bp(n)
        read_once_bp = generate_read_once_bp(n)
        ft_read_twice = fourier_transform(read_twice_bp, n)
        ft_read_once = fourier_transform(read_once_bp, n)
        results.append({
            "n": n,
            "ft_read_twice": ft_read_twice,
            "ft_read_once": ft_read_once
        })
    metric_value = sum(ft_read_twice) / len(n_values)
    instances_tested = len(results)
    conjecture_holds = all(abs(ft_read_twice) >= n**2 for n, _, _ in results)
    counterexample = "" if conjecture_holds else "read-twice BP meets  $O(n^2)$  bound"
    return {
        "metric_name": "Fourier Coefficient Sum",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 35)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"read-twice BP meets  $O(n^2)$  bound\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_362a060f.py", line 143, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_362a060f.py", line 119, in run_trial
    ft_read_twice = fourier_transform(read_twice_bp, n)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_362a060f.py", line 108, in fourier_tran
    representations = symmetric_group_representation(n)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_362a060f.py", line 93, in symmetric_gro
    representation[apply_permutation(cycle, [i])[0]][i] = sgn(cycle)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_362a060f.py", line 86, in apply_permuta
    return [bp[permutation[i]] for i in range(len(bp))]
           ~^^^^^^^^^^^^^^^^^^^^

```

IndexError: tuple index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing results, preventing evaluation of conjecture validity. | next: Debug and fix the symmetric_group_representation implementation to handle permutations correctly

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106899
2	preregistration	ollama_remote	qwen3:8b	0	0	24006
3	novelty	ollama_remote	qwen3:8b	0	0	20855
4	novelty	ollama_remote	qwen3:8b	0	0	15521
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41104
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17005
7	judge	ollama_remote	qwen3:8b	0	0	15058

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 240447 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/399f95beac0c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/399f95beac0c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/399f95beac0c.tar.gz` (if generated)